

STRUCTURED WRITING: A SYSTEM FOR TRANSFERRING IDEAS TO PAPER

by Rob Cameron

"Every novel should have a beginning, a muddle and an end" (Peter de Vries)

"Writing is easy; all you do is sit staring at a blank sheet of paper until the drops of blood form on your forehead" (Gene Fowler)

Seeing your articles, reports, dissertations or books in print is immensely satisfying but the process of writing them is hard, intellectual work. Sometimes you are obliged to write: a project report, instructions for a customer or a dissertation for a degree. Sometimes you want to write: for career advancement, to share your ideas or for the satisfaction of seeing your scholarship made permanent. Either way, presenting your thoughts in a way which will be received favourably by your readers demands a lot of effort. 'Structured writing' is one way to help with the process of organising your thoughts and hence finding the words to express them.

What is 'structured writing' ?

Structured writing is a systematic way of organising the content of any piece of written work. It will help you to write more efficiently, and will give you a basis for discussing your written work with others, *be/ore* you have spent a lot of time preparing the first draft of a paper, report or book. It is a simple but useful technique that I have developed over the last twenty or so years to help myself and many students to complete articles, and more especially dissertations, and to meet deadlines with least effort. It has been used by undergraduate students writing a report on a period of industrial training, by postgraduate students writing dissertations and theses for higher degrees (often in their second or third languages) and by staff and research visitors writing scholarly papers and articles for journal or conference publication.

Why use a structured approach to writing?

Writing reports and papers is definitely a skill which can be learned. As with any skill, watching an expert do it makes it look easy, but trying it yourself can seem extremely hard. This is partly because there are many *traps for the unwary*. Let's list some of these pitfalls.

1. *Not starting*: the empty piece of paper (or the blank computer screen) can be quite intimidating and it is easy to waste time staring at the screen, agonising over how to start.
2. *Wasting time writing the wrong paper*. Perhaps you spend several weeks labouring over 20 pages of a paper: *then* you find out that it is not what your boss, your co-authors (or even you!) really wanted.
3. *Writing an illogical presentation of the work* - '*a beginning, a muddle and an end*': there is a temptation to write the material of a paper in the order in which the work was done. This is usually *not* the best order in which to present the results. If you start on page 1 and 'make it

up as you go along' you will end up with a story like 'spaghetti' - it wanders all over the place and is c1ten tied in knots. A variation of this trap is due to the use of word-processors: it is *very* tempting to re-use files of text which have been written *for* another purpose, such as a student project, a conference presentation or an industrial report. A simplistic use of such material will only lead to an article like 'dish of the week' - a bit of everything all mixed together.

4. *Writing an unbalanced paper*: the first couple of sections are long and detailed, but as the paper progresses and enthusiasm wanes, the sections get shorter and shorter. First versions of a paper are often far too long, because it is tempting to try to explain *everything*.
5. *Writer's block*: You get held up at one point of the text and progress on the whole paper is stopped until the 'block' is cleared.

Why does structured writing help you to deal with *all* of these problems? It helps because there are **two** aspects to writing any document:

- o sorting out thoughts and ideas into a logical and coherent sequence, and
- o finding appropriate words to express the argument.

Usually we try to address these aspects simultaneously but then find ourselves switching between them, making only slow progress with the job as a whole. A better approach is to deal with the two tasks *independently*. 'Structured writing' is a method *for* working with the first task; when this is complete, the mind is free to concentrate on the second.

Background: how did we get here?

The inspiration for structured writing comes primarily from the field of computer programming, so it's appropriate to spend a little time reviewing changes in the practice of writing computer programs. Understanding the concept of structured programming will lead directly to an understanding of structured writing.

In the sixties and early seventies, when computer programming was becoming common practice among scientists and engineers, probably the most usual approach to writing programs was to take a stack of coding *forms* and fill them in line by line starting at the beginning and struggling through to the end. In operation, the computer would start with the first instruction, execute it, then follow with the second, third, etc. until the 'END' instruction was reached. Each instruction would be written in a 'high-level' language such as BASIC, FORTRAN or COBOL. This approach was an evolution from 'low-level' programming in assembly language or machine code, where the computer had to be told *exactly*, step by step, what operations to perform. The languages themselves encouraged this (sequential) style of writing: for example, in those days *every line* in a BASIC program *had* to be numbered. This approach is workable for small programs where the computer must execute a series of instructions in the correct order: such as most mathematical calculations.

There were three major driving forces behind the revolution in programming style which

has led eventually to the use of modem techniques. First of all, programmers naturally wanted to re-use fragments of programs they had written before. For example if you write a program which calculates and draws a graph based on certain data, it would be natural to use the same lines of program in another project if a graph were needed. Secondly, as programs became longer and more complex, a team of programmers would be needed to develop and test the software. It turned out to be difficult to co-ordinate the team if they were writing one long list of instructions: what was needed was the opportunity to break the program into a set of separate tasks which could be assigned to different members of the team, who would work on them independently. Finally, the development of the *WIMP* interface (Windows, Icons, Menus, Pointer) by Xerox, Apple and Microsoft introduced the idea that programs need not execute in the same order each time they run: applications such as word processors, file managers, electronic mail, etc. might be called and executed by the user in an arbitrary order.

Structured programming

During the seventies and eighties, the concept of *Structured Programming* evolved. Programs were considered at a higher, functional level, rather than at the instruction level. Before writing any program instructions, programmers were trained to break clown the overall function of a program into a set of independent tasks, and to break each task into a set of sub-tasks, etc., clown to a level where each sub-task represents only one well defined action. Then each 'action' is defined in terms of the data it needs as input, and the results it produces. This elementary software 'chip' can be prog-rammed, tested and saved in a library quite independently of all the other 'chips'. The overall program is assembled according to the hierarchical relationships between the tasks.

I want to illustrate this concept by considering a program to control the temperature of a computerised central heating system. *Readers - bear with me!* I have deliberately chosen an example which will (probably) be unfamiliar to you, because I believe it proves the power of the approach. Even though you might not nave any experience of control systems or of computer programming, 1 am confident you will be able to appreciate the *structure* of the program.

Suppose the program is going to control the temperature of a room and record the measured temperatures. At the highest level, the purpose of the program is to heat the room. Let's call this task 'HeatRoom'. To do ibis, the program must carry out three sub-tasks: call them 'StartUp', 'DoControl' and 'ShutDown'. if we focus on the second task, we can guess that it must have some sub-tasks of its own, such as: 'MeasureTemperature', 'AdjustHeaters', 'SaveData', 'CheckIffimeToStop'. Clearly these tasks can be broken clown further: adjusting the heaters will involve checking how warm the room is, calculating the best setting for the boiler, sending signals to the boiler, etc. Only when all the tasks and sub-tasks nave been identified and put in proper arrangement with each other does the programmer start to *write* the detailed instructions.

This process is summarised in Figure 1.

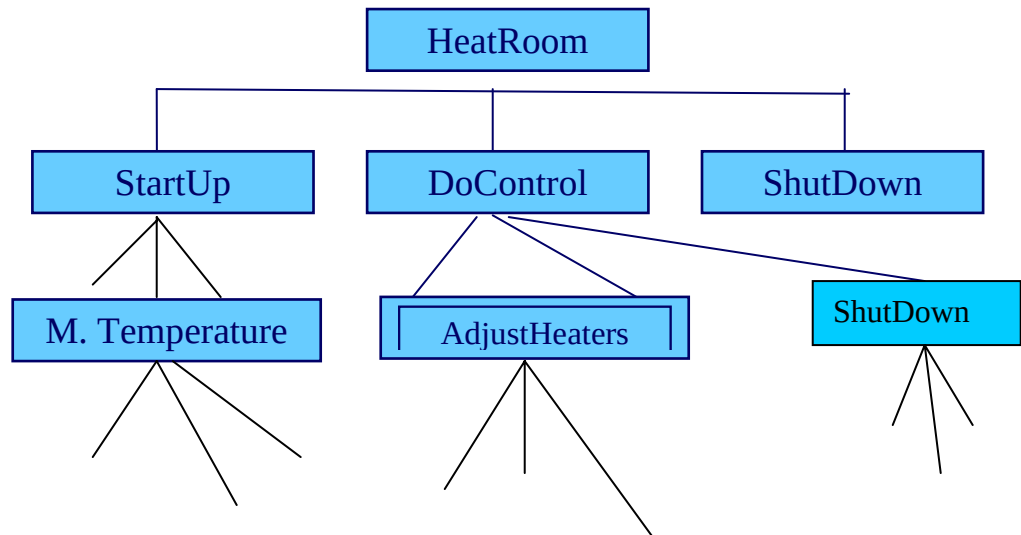


Figure 1

What conclusions can we draw from the hierarchy in Figure 1?

- o Even **without background knowledge** in programming, it is possible to follow the outline above and with so me practice, it is possible to break clown almost any activity in the same way (as an exercise, think about the tasks involved in getting up, having breakfast and going to work).
- o Notice that the program has a **logical structure imposed** on it from the start: *before* any programming language instructions have been written.
- o You can see that the program is composed of a set of sub-tasks which are independent of each other. These sub-tasks could be given to different programmers to work en; or, if one person is writing the program perhaps they could start with the task to adjust the heaters, even though it takes place in the *middle* of the program.
- o The completed outline can be discussed by the project team and the customer,*before* any major programming effort has been expended.
- o Notice how easy it would be to slot in a ready-made fragment of program (for example to save the data) if this was already available from another source.

Structured writing

What has this to do with writing an article or a report? I advocate adopting the same, logical approach to *writing* as to *programming*.

Let's suppose you have finished a major project and it's time to write the final report. Structured writing of the report follows these steps:

1. Collect together the **main points** you want to make, grouped into coherent sets of ideas.
2. Draw up the **outline**: a hierarchical list of contents, with the title at the top of the tree, followed by the section headings, sub-section headings, sub-subsection headings, etc, clown to the level of paragraphs.
3. Assign a nominal length (in numbers of words or pages, as appropriate) to each **section** and sub-section (the *(sub-)section* takes the place of the *(sub-) task* discussed in the programming case). By deciding on the length of every section at the structuring stage you will produce a final version which is the length you want and is well balanced, giving an amount of space to each topic merited by its significance.
4. **Consider** the outline carefully: does it represent the most logical way of presenting the information you want to convey?. Is the estimated overall length about right? Is the length of each section appropriate?
5. **Refine** the outline in the light of discussion and feedback from supervisor or colleagues.
6. Fill in the text for each section, using the nominal length as a target. This can be done by writing from scratch, re-using text and figures from earlier reports or asking colleagues to write about their own contribution to the project within the defined parameters of the outline.

The **outline** is essentially similar to the list of contents you would write for a book. However, in the finished article or report not all the sub-sections you have identified in the outline will appear as numbered sections or paragraphs - only the top two or three levels will be used in that way. Furthermore, writing the information in the form of a tree rather than a list makes it easier to assess the form and structure of the work *as a whole*. You tend to read a list sequentially, from top to bottom: the purpose of the outline is to present an overview.

A **practical point** is that you will either need a wide sheet of paper to draw the complete outline, or you will have to break the tree into sub-trees, e.g. one tree for each section.

Figure 2 shows a partial tree, with a sub-tree in Figure 2a, for a consultancy report on production systems for *Widget International, SA*.

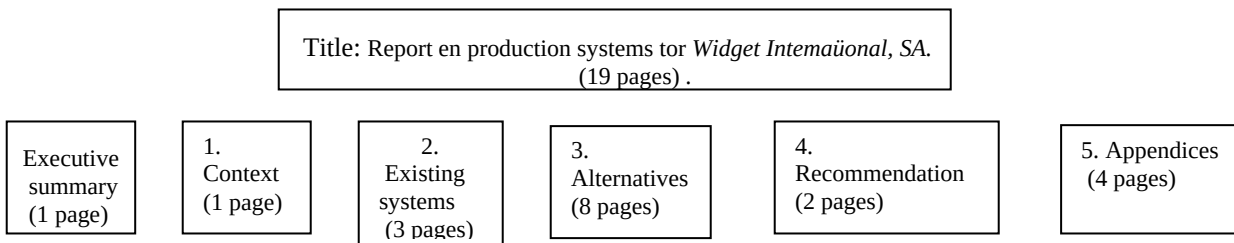


Figure 2

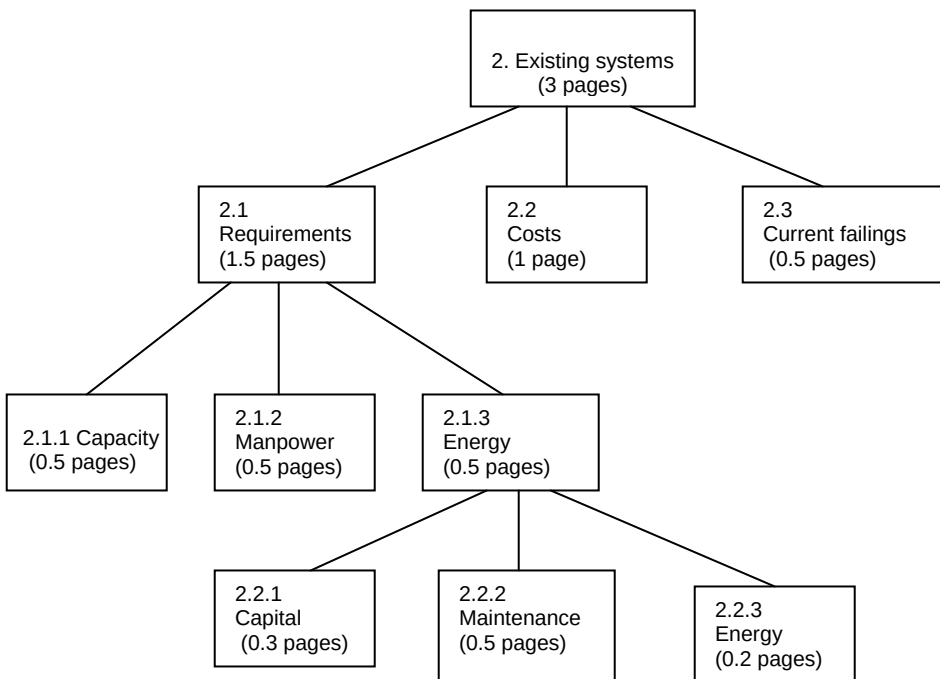


Figure 2a

The pros and cons

Let's return to our list of pitfalls and see how structured writing helps.

1. *Not starting*: with the outline in hand, you are faced with the problem of writing a number of small pieces of text – not the intimidating problem of how to write the complete work. It is often clearer where and how to begin.
2. *Wasting time writing the wrong paper*: use the outline to discuss with colleagues or your supervisor the content and organisation of the paper. That way you can get the composition of the article right *before* you write it.
3. *Writing an illogical presentation of the work*: you can see at a glance the logic (or lack of it) in your outline. It is easier to see how and where you can re-use existing text. You won't end up with 'spaghetti' or 'dish of the week'.
4. *Writing an unbalanced paper*: you balance the paper at the outset. Each section is written to the target lengths decided at the outline stage.
5. *Writer's block*: if you get blocked in one (sub-)section there are several others to work with. The chances are that when you return to the difficult section, the block will have disappeared.

Nothing comes free: what are the costs? When the overall article is completed, there are two possible problems:

- . the writing style might vary from one section to another. This is more likely if you *do* re-use text from other sources or other writers.
- . the arguments in the paper might not *flow* smoothly; the overall effect might be rather mechanical, like 'robot-speak'.

Neither of these problems is serious. First, just being aware of them will make you more careful to avoid them. Second, writing is an *iterative* process: you will write and revise, write and revise, probably several times. Each time around, you should try to get feedback from others, for example by using the *Action Learning* approach described by Robert Brown in an earlier article. Variations in style and readability can be picked up and eliminated at this stage.

Finally, make no mistake: presenting a mass of (often complex) ideas in a form which others can understand and value is hard, intellectual work. Structured writing helps with the task of **sorting out thoughts into** a logical and coherent sequence: in my experience this is the harder of the two aspects of writing which were mentioned at the start. It has to be done at some time in the process of writing; structured writing forces it to be done 'up front'. It takes a lot of self-discipline to keep working at the outline when you would rather be getting on with the easier task of putting words to paper.

Conclusion

I'm sure that most of us do some structuring when writing, but usually don't follow it through. By accepting the discipline of structured writing, I believe that it will impose clarity of argument, logical presentation and conciseness on your, article as a whole. The *quality* of writing can be improved by repeated refinement, but if the ideas in an article are not presented in a coherent way, then no amount of elegant prose will redeem it.

Further reading

Brown, R., 'Write right first time', *Literati Newslines* Special Issue, 1994.

Day, R. A., 'How to write and publish a scientific paper', 4th edition, Cambridge University Press, 1994

Mitchell, J., 'How to write reports', Fontana, 1974.